

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Практическая работа 1

По дисциплине Проектирование и реализация баз данных

Тема работы Установка СУБД PostgreSQL, создание базы данных в pgAdmin, создание и модификация структуры базы данных HR в PostgreSQL с использованием графического интерфейса pgAdmin и языка SQL.

Обучающийся Рекун Ирина Данииловна

Факультет прикладной информатики

Группа K3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Рекун И.Д

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Войтюк Т.Е

(Ф.И.О.)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
Задание 1.....	3
Задание 2.....	7
Задание 3.....	12
Задание 4.....	13
Задание 5.....	15
Задание 6.....	17
Выводы и анализ результатов работы.....	19

ВВЕДЕНИЕ

Цель работы: Изучение основных возможностей языка SQL по выборке и обработке данных, включая применение операторов SELECT, WHERE, ORDER BY, числовых и символьных функций, функций работы с датами, а также условных выражений CASE и COALESCE. Закрепление навыков формирования корректных SQL-запросов и анализа их результатов.

Задачи, решаемые при выполнении работы:

1. Освоить синтаксис оператора SELECT и научиться выполнять простые и расширенные выборки данных.
2. Научиться отображать структуру таблиц и формировать выборки с переименованием столбцов, сортировкой и удалением дублирующихся строк.
3. Отработать применение условий выборки с использованием оператора WHERE, диапазонов, списков значений и проверки на NULL.
4. Освоить работу с числовыми функциями (ROUND, арифметические выражения) для вычисления новых значений.
5. Научиться использовать символьные функции и операторы конкатенации для формирования текстовых выводов.
6. Применить функции обработки дат и извлечения отдельных частей даты в SQL-запросах.
7. Изучить использование условных выражений CASE и COALESCE для реализации логики внутри запросов.
8. Закрепить навыки анализа ошибок в SQL-запросах и формирования корректных решений.

Задание 1. Описание структуры таблицы, выборка данных из таблицы, задание имён столбцов, сортировка строк с помощью предложения ORDER BY.

1.1 Команда

```
SELECT *  
FROM "EmployeesDepartments". JOB_GRADES;
```

не будет выполнена успешно из-за синтаксической ошибки: между названием схемы и названием таблицы стоит пробел, а в SQL после точки имя объекта должно идти без пробела.

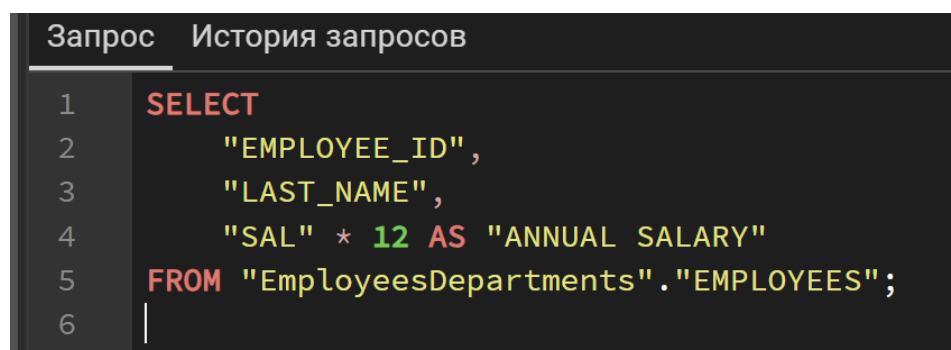
1.2 Команда

```
SELECT "EMPLOYEE_ID", "LAST_NAME"  
"SAL" × 12 ANNUAL SALARY  
FROM "EmployeesDepartments"."EMPLOYEES";
```

содержит следующие ошибки:

1. Между "LAST_NAME" и "SAL" отсутствует запятая.
2. Выражение "SAL" × 12 синтаксически неверно. Необходимо использовать только "*" так как оператора "×" не существует в SQL.
3. Алиас ANNUAL SALARY записан неверно так как имена столбцов с пробелами должны быть в кавычках.
4. Выражение "SAL" × 12 и алиас ANNUAL SALARY записаны без корректного разделителя (например, AS) из-за чего SQL не может определить границу между выражением и алиасом, что приводит к синтаксической ошибке.

Команда SELECT после исправления 4 ошибок представлена на рисунке 1.1.



The screenshot shows a SQL query editor with a dark background. At the top, there are two tabs: 'Запрос' (Query) and 'История запросов' (Query History). The 'Запрос' tab is active. Below the tabs, a SQL query is displayed on a line-by-line basis, numbered 1 through 6 in the left margin. The query is:
1 SELECT
2 "EMPLOYEE_ID",
3 "LAST_NAME",
4 "SAL" * 12 AS "ANNUAL SALARY"
5 FROM "EmployeesDepartments"."EMPLOYEES";
6 |

Рисунок 1.1 – Исправленная команда SELECT.

1.3 Необходимо написать запрос, который отображает структуру таблицы DEPARTMENTS (Рисунок 1.2).

column_name	is_nullable	Type
DEPARTMENT_ID	NO	smallint
DEPARTMENT_NAME	NO	character varying
MANAGER_ID	YES	integer
LOCATION_ID	YES	smallint

Рисунок 1.2 – Структура таблицы DEPARTMENS

Запрос, который отображает структуру таблицы DEPARTMENS отображен на рисунке 1.3.

```

Запрос  История запросов
1  SELECT
2      column_name,
3      is_nullable,
4      data_type
5  FROM information_schema.columns
6  WHERE table_name = 'departments';
7

```

Рисунок 1.3 – Запрос для отображения структуры таблицы DEPARTMENS

Далее необходимо было сформировать запрос на выборку данных из таблицы DEPARTMENS. Сформулированный запрос показан на рисунке 1.4.

```

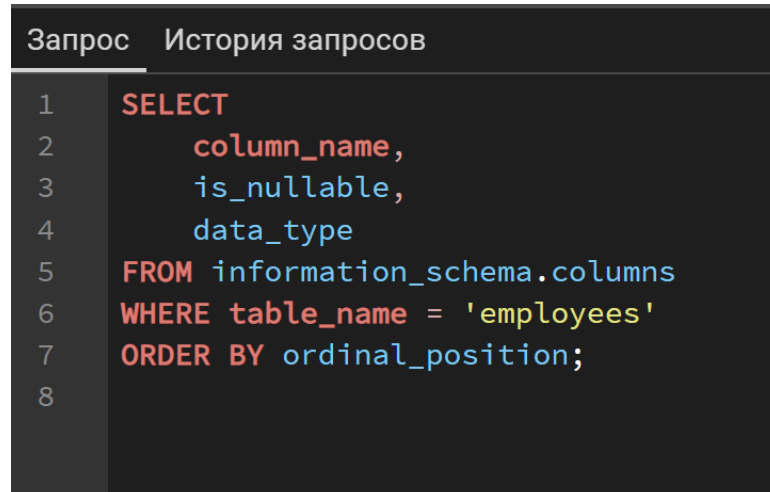
Запрос  История запросов
1  SELECT
2      "DEPARTMENT_ID",
3      "DEPARTMENT_NAME",
4      "MANAGER_ID",
5      "LOCATION_ID"
6  FROM "DEPARTMENTS"
7  ORDER BY "DEPARTMENT_ID";
8

```

Рисунок 1.4 – Запрос на выборку данных из таблицы DEPARTMENS.

1.4 Необходимо написать запрос, который отображает структуру таблицы EMPLOYEES, представленную в методичке.

Сформулированный запрос, отображающий структуру таблицы EMPLOYEES, представлен на рисунке 1.5.



The image shows a screenshot of a SQL query editor with a dark background. At the top, there are two tabs: 'Запрос' (Query) and 'История запросов' (Query History). The 'Запрос' tab is active. Below the tabs, a SQL query is written in a light-colored font. The query is as follows:

```
1  SELECT
2      column_name,
3      is_nullable,
4      data_type
5  FROM information_schema.columns
6  WHERE table_name = 'employees'
7  ORDER BY ordinal_position;
8
```

Рисунок 1.5 – Запрос для отображения структуры EMPLOYEES.

Запрос выводит список всех столбцов таблицы EMPLOYEES, их типы и признак допуска NULL, что соответствует структуре, приведённой на рисунке в задании.

1.5 В данном задании необходимо было вывести основные сведения о каждом сотруднике: номер сотрудника, фамилию, идентификатор должности и дату найма. При этом номер сотрудника должен быть указан первым. Для выполнения задания был сформулирован SQL-запрос, который выбирает соответствующие столбцы из таблицы EMPLOYEES (Рисунок 1.6). Для того, чтобы номер сотрудника отображался первым, соответствующий столбец EMPLOYEE_ID был указан первым в списке выбираемых полей оператора SELECT.

```
Запрос  История запросов
1  SELECT
2      "EMPLOYEE_ID",
3      "LAST_NAME",
4      "JOB_ID",
5      "HIRE_DATE"
6  FROM "EMPLOYEES"
7  ORDER BY "EMPLOYEE_ID";|
8
```

Рисунок 1.6 – Запрос для вывода фамилии каждого служащего, должности, даты найма и номера.

1.6 В данном задании требуется вывести список всех должностей без повторов из таблицы EMPLOYEES. Для этого используем оператор DISTINCT, который удаляет дублирующиеся значения в выбранном столбце. Сформулированный запрос представлен на рисунке 1.7.

```
Запрос  История запросов
1  SELECT DISTINCT "JOB_ID"
2  FROM "EMPLOYEES"
3  ORDER BY "JOB_ID";
4
```

Рисунок 1.7 – Запрос для вывода неповторяющихся должностей

В этом запросе столбец "JOB_ID" выбирается из таблицы EMPLOYEES, а DISTINCT оставляет только уникальные идентификаторы должностей. Оператор ORDER BY "JOB_ID" выполняет сортировку результата по должности, что делает вывод упорядоченным и соответствующим примеру.

1.7 В данном задании необходимо вывести на экран фамилию, соединённую с идентификатором должности через запятую и пробел. А также назвать новый столбец Employee and Title. Для реализации соединения строк используется оператор конкатенации "||", применяемый в PostgreSQL для объединения текстовых значений. Сформулированный запрос представлен на рисунке 1.8.

Запрос	История запросов
1	SELECT
2	"LAST_NAME" ', ' "JOB_ID" AS "Employee and Title"
3	FROM "EMPLOYEES";
4	

Рисунок 1.8 – Запрос для вывода фамилии, соединённой с идентификатором должности

В данном запросе первая часть выражения берёт фамилию, затем добавляется строковый литерал ', ', создающий запятую и пробел, а после чего присоединяется значение столбца JOB_ID. Оператор AS присваивает результирующему выражению имя "Employee and Title".

Задание 2. Выборка данных и изменение последовательности вывода строк, ограничение количества возвращаемых строк с помощью предложения WHERE, сортировка строк с помощью предложения ORDER BY

2.1 В данном задании необходимо было вывести фамилии и заработные платы сотрудников, чья зарплата превышает 12000. Для выполнения этого условия используем оператор WHERE, позволяющий ограничить набор выводимых строк согласно логическому выражению. Запрос для вывода фамилии и заработной платы служащих представлен на рисунке 1.9.

1	SELECT
2	"LAST_NAME",
3	"SALARY"
4	FROM "EMPLOYEES"
5	WHERE "SALARY" > 12000;

Рисунок 1.9 – Запрос для вывода фамилии и заработной платы служащих.

В данном запросе столбцы LAST_NAME и SALARY выбираются из таблицы EMPLOYEES, а условие WHERE "SALARY" > 12000 обеспечивает вывод только тех сотрудников, которые зарабатывают больше указанного значения.

2.2 Необходимо создать запрос для того, чтобы вывести фамилию и номер отдела служащего под номером 176. Сформулированный запрос представлен на рисунке 1.10.


```
Запрос  История запросов
1  SELECT
2      "LAST_NAME",
3      "DEPARTMENT_ID"
4  FROM "EMPLOYEES"
5  WHERE "EMPLOYEE_ID" = 176;
6
```

Рисунок 1.10 – Запрос для служащего под номером 176.

Для выборки конкретной строки используем условие WHERE, позволяющее задать условие равенства значения первичного ключа, то есть выбирающее ровно одну строку – сотрудника, у которого идентификатор равен 176. Таким образом, выводятся только необходимые столбцы LAST_NAME и DEPARTMENT_ID.

2.3 В данном задании необходимо изменить запрос из задания 2.1 и вывести фамилии и оклады всех служащих, чей оклад не входит в диапазон от 5000 до 12000. Измененный запрос показан на рисунке 1.11.

```
1  SELECT
2      "LAST_NAME",
3      "SALARY"
4  FROM "EMPLOYEES"
5  WHERE "SALARY" NOT BETWEEN 5000 AND 12000;
6
```

Рисунок 1.11 – Запрос для служащих, чей оклад не входит в диапазон от 5000 до 12000.

Для проверки принадлежности значения диапазону используется оператор BETWEEN, а для получения противоположного результата – его отрицание NOT BETWEEN. Таким образом, оператор NOT BETWEEN исключает из результата строки, где SALARY находится внутри диапазона.

2.4 Требуется вывести фамилию, идентификатор должности и дату начала работы сотрудников, принятых на работу с 16.02.2011 по 12.05.2011. И отсортировать данные в порядке возрастания даты найма. Запрос для выполнения этого задания представлен на рисунке 1.12.

```

1  SELECT
2      "LAST_NAME",
3      "JOB_ID",
4      "HIRE_DATE"
5  FROM "EMPLOYEES"
6  WHERE "HIRE_DATE" BETWEEN '2011-02-16' AND '2011-05-12'
7  ORDER BY "HIRE_DATE";

```

Рисунок 1.12 – Запрос для служащих, нанятых в период с 16 февраля 2011 по 12 мая 2011 г

Для фильтрации по дате используется оператор BETWEEN, который позволяет нам проверить попадание даты найма в указанный диапазон. Оператор ORDER BY используется для сортировки вывода данных в порядке возрастания даты найма.

2.5 В данном задании необходимо вывести фамилии и номера отделов сотрудников, работающих в подразделениях 20 и 50. Созданный запрос представлен на рисунке 1.13.

```

1  SELECT
2      "LAST_NAME",
3      "DEPARTMENT_ID"
4  FROM "EMPLOYEES"
5  WHERE "DEPARTMENT_ID" IN (20, 50)
6  ORDER BY "LAST_NAME";
7  SELECT
8      "LAST_NAME",
9      "DEPARTMENT_ID"
10 FROM "EMPLOYEES"
11 WHERE "DEPARTMENT_ID" IN (20, 50)
12 ORDER BY "LAST_NAME";
13

```

Рисунок 1.13 – Запрос для служащих из отделов 20 и 50

Условие WHERE ограничивает результат теми строками, где DEPARTMENT_ID равен 20 или 50. Поскольку требуется упорядочить данные по фамилиям в алфавитном порядке, применяем оператор ORDER BY "LAST_NAME".

2.6 В данном задании необходимо было изменить запрос из задания 2.3 для вывода фамилий и окладов служащих отделов 20 и 50, зарабатывающих от 5000 до 12000 и назвать столбцы Employee и Mounthly Salary, соответственно. Запрос для выполнения этого задания представлен на рисунке 1.14.

```

1  SELECT
2      "LAST_NAME" AS "Employee",
3      "SALARY" AS "Monthly Salary"
4  FROM "EMPLOYEES"
5  WHERE "DEPARTMENT_ID" IN (20, 50)
6      AND "SALARY" BETWEEN 5000 AND 12000
7  ORDER BY "LAST_NAME";

```

Рисунок 1.14 – Запрос для служащих отделов 20 и 50, зарабатывающих от 5000 до 12000

Выбираем фамилии и оклады сотрудников отделов 20 и 50, при этом в выборку включаем только те строки, у которых значение SALARY находится в диапазоне от 5000 до 12000. Для проверки принадлежности оклада диапазону используем оператор BETWEEN. Столбцы переименовываем с помощью оператора AS, что позволяет нам вывести их под заголовками Employee и Monthly Salary.

2.7 В данном задании необходимо было вывести фамилии и должности тех сотрудников, которые не имеют менеджера. То есть требуется выбрать строки, где значение столбца MANAGER_ID отсутствует. SQL - запрос для служащих, не имеющих менеджера представлен на рисунке 1.15.

```

1  SELECT
2      "LAST_NAME",
3      "JOB_ID"
4  FROM "EMPLOYEES"
5  WHERE "MANAGER_ID" IS NULL;

```

Рисунок 1.15 – Запрос для служащих, не имеющих менеджера.

Используем оператор IS NULL, позволяющий отобрать только те записи, у которых поле MANAGER_ID не содержит значения. В результате выводятся сотрудники, не закреплённые ни за одним менеджером.

2.8 В данном задании требуется вывести фамилии сотрудников и их комиссионные выплаты. Также требуется отсортировать результат по убыванию заработной платы и суммы комиссионных. На рисунке 1.16 отображен запрос для выполнения этого задания.

```

1  SELECT
2      "LAST_NAME",
3      "COMMISSION_PCT",
4      "SALARY"
5  FROM "EMPLOYEES"
6  WHERE "COMMISSION_PCT" IS NOT NULL
7  ORDER BY "SALARY" DESC, "COMMISSION_PCT" DESC;

```

Рисунок 1.16 – Запрос для служащих, зарабатывающих комиссионные.

Оператор ORDER BY выполняет сортировку сначала по окладу, затем по комиссионным выплатам, оба раза в порядке убывания. А условие WHERE "COMMISSION_PCT" IS NOT NULL помогает вывести только тех сотрудников, которые получают комиссионные.

2.9 В данном задании необходимо было вывести фамилии тех сотрудников, у которых третья буква в фамилии является буквой “а”. При этом запрещено использовать строковые функции, поэтому в качестве инструмента фильтрации применим оператор сравнения LIKE, который не является функцией (Рисунок 1.17).

```

1  SELECT
2      "LAST_NAME"
3  FROM "EMPLOYEES"
4  WHERE "LAST_NAME" LIKE ' __a%';
5

```

Рисунок 1.17 – Запрос для вывода фамилий, в которых третья буква – а.

Шаблон ' __a%' задаёт необходимую структуру строки: первые два символа могут быть любыми, третья буква должна быть 'а', а далее может следовать любая последовательность символов. Такой подход позволяет нам выполнить условие без применения строковых функций.

2.10 Необходимо вывести фамилии, должности и оклады всех служащих, работающих торговыми представителями (SA_REP) или клерками на складе (ST_CLERK), при этом их оклад не должен совпадать с одним из следующих значений: 2600, 3100, 8600 или 11000. Созданный запрос представлен на рисунке 1.18.

```

1  SELECT
2      "LAST_NAME",
3      "JOB_ID",
4      "SALARY"
5  FROM "EMPLOYEES"
6  WHERE "JOB_ID" IN ('SA_REP', 'ST_CLERK')
7      AND "SALARY" NOT IN (2600, 3100, 8600, 11000)
8  ORDER BY "LAST_NAME";

```

Рисунок 1.18 – Запрос для служащих, работающих торговыми представителями или клерками на складе.

Условие IN отбирает только нужные типы должностей, а конструкция NOT IN исключает оклады 2600, 3100, 8600 и 11000.

2.11 В данном задании необходимо было изменить запрос из задания 2.8 таким образом, чтобы отобрать только тех сотрудников, чьи комиссионные выплаты составляют 20% или более. Требовалось вывести фамилию, оклад и значение столбца COMMISSION_PCT, а также сохранить сортировку по убыванию оклада и комиссионных. Сформулированный запрос представлен на рисунке 1.19.

```

1  SELECT
2      "LAST_NAME",
3      "SALARY",
4      "COMMISSION_PCT"
5  FROM "EMPLOYEES"
6  WHERE "COMMISSION_PCT" IS NOT NULL
7      AND "COMMISSION_PCT" >= 0.20
8  ORDER BY "SALARY" DESC, "COMMISSION_PCT" DESC;

```

Рисунок 1.19 – Запрос для служащих, чьи комиссионные выплаты составляют 20% или более.

В запросе используются два условия фильтрации: IS NOT NULL позволяет исключить строки без комиссионных, а выражение COMMISSION_PCT >= 0.20 выбирает только тех сотрудников, чей процент комиссионных равен или превышает 20.

Задание 3. Составление запросов, требующих использования числовых функций (TRUNC, ROUND и т. д.)

3.1 В данном задании необходимо было вычислить новый оклад для каждого сотрудника, увеличив его на 9,5%, и округлить результат до целого значения. Также требовалось вывести номер сотрудника, фамилию, исходный оклад и новый оклад под заголовком New Salary. На рисунке 1.20 отображен запрос для выполнения этого задания.

```
1  SELECT
2      "EMPLOYEE_ID",
3      "LAST_NAME",
4      "SALARY",
5      ROUND("SALARY" * 1.095) AS "New Salary"
6  FROM "EMPLOYEES";
```

Рисунок 1.20 – Запрос для вывода номера служащего, его фамилии, оклада и нового оклада, повышенного на 9,5%.

В этом запросе вычисление нового оклада выполняется путём умножения значения SALARY на коэффициент 1.095, что соответствует повышению на 9,5%, а функцию ROUND применяем для округления полученного результата до ближайшего целого числа.

3.2 В данном задании необходимо было дополнить запрос из прошлого задания вычислением разницы между новым и старым окладом, то есть требуется вывести ещё один столбец, отражающий величину повышения. Разницу рассчитаем как вычитание первоначального оклада из нового значения, полученного путём увеличения SALARY на 9,5% и последующего округления (Рисунок 1.21).

```
1  SELECT
2      "EMPLOYEE_ID",
3      "LAST_NAME",
4      "SALARY",
5      ROUND("SALARY" * 1.095) AS "New Salary",
6      ROUND("SALARY" * 1.095) - "SALARY" AS "Increase"
7  FROM "EMPLOYEES";
```

Рисунок 1.21 – Запрос для вывода нового оклада и размера повышения

Задание 4. Составление запросов, требующих символьных функций (INITCAP, POSITION, LENGTH, CONCAT и т.д.)

4.1 В этом задании необходимо было преобразовать каждую строку таблицы EMPLOYEES в текстовую строку, объединяющую все столбцы через запятую. Для выполнения такой задачи требуется приведение значений к текстовому

типу и их последовательное соединение оператором конкатенации. Запрос показан на рисунке 1.22.

```
1  SELECT
2      "EMPLOYEE_ID"::text || ', ' ||
3      "FIRST_NAME"::text || ', ' ||
4      "LAST_NAME"::text || ', ' ||
5      "EMAIL"::text || ', ' ||
6      "PHONE_NUMBER"::text || ', ' ||
7      "HIRE_DATE"::text || ', ' ||
8      "JOB_ID"::text || ', ' ||
9      "SALARY"::text || ', ' ||
10     COALESCE("COMMISSION_PCT"::text, 'NULL') || ', ' ||
11     COALESCE("MANAGER_ID"::text, 'NULL') || ', ' ||
12     COALESCE("DEPARTMENT_ID"::text, 'NULL')
13     AS "THE_OUTPUT"
14 FROM "EMPLOYEES";
```

Рисунок 1.22 – Запрос для объединения всех полей таблицы EMPLOYEES в одну строку

Каждый столбец приводится к текстовому типу при помощи оператора `::text`, а затем объединяется с соседними значениями через запятую и пробел. Для корректного отображения пустых значений используется `COALESCE`, подставляющий строку 'NULL'.

4.2 В этом задании требовалось вывести фамилии тех сотрудников, чьи фамилии начинаются с букв J, A или M, при этом фамилия должна отображаться в формате с заглавной первой буквой и строчными остальными символами. Дополнительно необходимо было показать длину каждой фамилии. На рисунке 1.23 отображен запрос для вывода отформатированных фамилий и их длины.

```
1  SELECT
2      INITCAP("LAST_NAME") AS "Last Name",
3      LENGTH("LAST_NAME") AS "Length"
4  FROM "EMPLOYEES"
5  WHERE "LAST_NAME" LIKE 'J%'
6         OR "LAST_NAME" LIKE 'A%'
7         OR "LAST_NAME" LIKE 'M%'
8  ORDER BY "LAST_NAME";
```

Рисунок 1.23 – Запрос для вывода отформатированных фамилий и их длины

Функция `INITCAP` приводит значение столбца `LAST_NAME` к виду «Первая буква строчная», что делает вывод более читаемым. Функция `LENGTH` вычисляет количество символов в фамилии. Условие `WHERE` с

оператором LIKE отбирает только те строки, где фамилия начинается на J, A или M. Оператор ORDER BY упорядочивает результат по фамилии.

4.3 В этом задании требовалось вывести информацию о сотрудниках в следующем виде: <фамилия> зарабатывает <оклад> в месяц, но желает <утроенный оклад>. Необходимо было объединить фамилию, текстовые фрагменты и числовые данные в один результирующий столбец с названием Dream Salaries. Запрос для формирования строки с желаемым окладом изображен на рисунке 1.24.

```
1  SELECT
2      "LAST_NAME" || ' зарабатывает ' ||
3      "SALARY"::text || ' в месяц, но желает ' ||
4      ("SALARY" * 3)::text AS "Dream Salaries"
5  FROM "EMPLOYEES";
```

Рисунок 1.24 – Запрос для формирования строки с желаемым окладом.

В запросе используется оператор конкатенации ||, который объединяет текстовые элементы в единую строку. Числовые значения SALARY и SALARY * 3 приводятся к текстовому типу с помощью ::text, чтобы корректно соединяться со строками.

Задание 5. Составление запросов, требующих функций для работы с датами и функции преобразования типов

5.1 В данном задании требовалось вывести текущую системную дату с присвоением результирующему столбцу имени Date. Для получения текущей даты в PostgreSQL используем функция CURRENT_DATE, которая возвращает значение в формате типа date (Рисунок 1.25).

```
Запрос  История запросов
1  SELECT
2      CURRENT_DATE AS "Date";
3
```

Рисунок 1.25 – Запрос для вывода текущей даты

Функция CURRENT_DATE извлекает сегодняшнюю дату из системы, а оператор AS задаёт удобочитаемое имя столбца.

5.2 В этом задании необходимо было вывести фамилии сотрудников и даты их найма, отобрав только тех работников, которые были приняты в 2011 году.

Для определения года используем функция EXTRACT, позволяющая получить нужную часть даты. Код запроса изображен на рисунке 1.26.

```
1 SELECT
2     "LAST_NAME",
3     "HIRE_DATE"
4 FROM "EMPLOYEES"
5 WHERE EXTRACT(YEAR FROM "HIRE_DATE") = 2011
6 ORDER BY "HIRE_DATE";
7
```

Рисунок 1.26 – Запрос для вывода сотрудников, нанятых в 2011 году

Функция EXTRACT(YEAR FROM HIRE_DATE) возвращает числовое значение года из даты найма. Условие фильтрации выбирает строки, в которых этот год равен 2011. Дополнительная сортировка по дате найма делает результат упорядоченным и наглядным.

5.3 В данном задании требовалось определить, сколько месяцев прошло с момента найма каждого сотрудника до текущей даты, и вывести эту информацию вместе с фамилией и датой найма. Расчёт выполняется с использованием извлечения года и месяца из дат. Сформулированный запрос представлен на рисунке 1.27.

```
1 SELECT
2     "LAST_NAME",
3     "HIRE_DATE",
4     (
5         (EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM "HIRE_DATE")) * 12 +
6         (EXTRACT(MONTH FROM CURRENT_DATE) - EXTRACT(MONTH FROM "HIRE_DATE"))
7     ) AS "MONTH_WORKED"
8 FROM "EMPLOYEES"
9 ORDER BY "MONTH_WORKED";
```

Рисунок 1.27 – Запрос для вычисления количества месяцев работы сотрудника

Разница между датами рассчитываем в месяцах: сначала определяем разницу в годах и переводим в месяцы, затем к ней добавляем разницу в месяцах. Итоговый столбец MONTH_WORKED отражает количество месяцев, отработанных сотрудником, а сортировка распределяет сотрудников по возрастанию стажа.

5.4 В данном задании необходимо было вывести фамилию сотрудника, дату его найма и день недели, в который он был принят на работу. Для получения дня недели используем функция TO_CHAR, которая позволяет преобразовать дату в строку с заданным форматом (Рисунок 1.28).

```

1  SELECT
2      "LAST_NAME",
3      "HIRE_DATE",
4      TO_CHAR("HIRE_DATE", 'Day') AS "DAY"
5  FROM "EMPLOYEES"
6  ORDER BY "HIRE_DATE";
7

```

Рисунок 1.28 – Запрос для вывода дня недели найма сотрудников.

Функция `TO_CHAR("HIRE_DATE", 'Day')` преобразует значение даты в словесное название дня недели. Оператор `ORDER BY` сортирует сотрудников по возрастанию даты найма.

Задание 6. Составление запросов, требующих применения условных выражений

6.1 В данном задании требовалось вывести фамилии сотрудников и информацию о комиссионных выплатах. Если сотрудник не получает комиссионных, вместо пустого значения необходимо вывести строку «No Commission». Для этого используем функция `COALESCE` с преобразованием типов, поскольку одно из возможных значений является текстом. На рисунке 1.29 отображен запрос для вывода комиссионных с заменой отсутствующих значений.

```

SELECT
    "LAST_NAME",
    COALESCE("COMMISSION_PCT"::text, 'No Commission') AS "COMM"
FROM "EMPLOYEES";

```

Рисунок 1.29 – Запрос для вывода комиссионных с заменой отсутствующих значений.

Функция `COALESCE` возвращает первое ненулевое значение из списка аргументов. Поэтому, если `COMMISSION_PCT` содержит числовое значение, оно преобразуется в текст и отображается в результатах. Если же значение равно `NULL`, подставляется строка «No Commission».

6.2 В данном задании требовалось определить разряд каждой должности сотрудника на основе таблицы соответствий, приведённой в условии. Для реализации классификации используется оператор `CASE`, позволяющий задать несколько условий и вернуть нужный результат в зависимости от значения поля `JOB_ID`. Сформулированный запрос представлен на рисунке 1.30.

```
1  SELECT
2      "JOB_ID",
3      CASE
4          WHEN "JOB_ID" IN ('AD_PRES', 'AD_VP', 'FI_MGR') THEN 'A'
5          WHEN "JOB_ID" IN ('IT_PROG', 'SA_MAN', 'FI_ACCOUNT', 'PU_MAN') THEN 'B'
6          WHEN "JOB_ID" IN ('ST_CLERK', 'SA_REP', 'PU_CLERK') THEN 'C'
7          ELSE 'Unknown'
8      END AS "GRADE"
9  FROM "EMPLOYEES";
```

Рисунок 1.30 – Запрос с использованием CASE для определения разряда должности

Выражение CASE последовательно проверяет значение JOB_ID и присваивает ему разряд, соответствующий таблице должностей. Такой подход позволяет выполнить категоризацию должностей в соответствии с примером

Выводы и анализ результатов работы.

В ходе выполнения практической работы были успешно решены все поставленные задачи: выполнен анализ структуры таблиц, составлены запросы различного уровня сложности, использованы средства фильтрации данных, сортировки, переименования столбцов, удаления дубликатов, а также функции обработки числовых, символьных значений и дат. Было подробно изучено поведение оператора SELECT в PostgreSQL. При выполнении заданий была сформирована серия SQL-запросов, демонстрирующих использование базовых операторов до более сложных конструкций, включая CASE, COALESCE, выражения с конкатенацией строк, арифметические вычисления, преобразование типов и использование EXTRACT для анализа дат. Также был выполнен разбор ошибок в исходных запросах, что позволило лучше понять требования синтаксиса PostgreSQL и избежать типичных ошибок при работе с кавычками, алиасами, операторами и типами данных.

Некоторые трудности возникли при работе с функциями обработки дат и при вычислении количества месяцев между датой найма и текущей датой, поскольку PostgreSQL не предоставляет прямого аналога функции MONTHS_BETWEEN. Эта проблема была успешно решена путём ручного составления формулы, использующей разницу между годами и месяцами. Дополнительные сложности сопровождали задания, связанные с объединением нескольких столбцов в одну строку, где требовалось учитывать правила преобразования типов и обработки NULL-значений. Для этого была использована функция COALESCE, а также явное приведение данных к типу text.

Благодаря выполнению всех заданий была достигнута основная цель работы: освоение практические приёмы формирования SQL-запросов, анализа результатов и обработки данных в PostgreSQL.